

PPO Parameters Reference (Current Implementation)

Generated on 2026-07-23.

1) Objective

For each trajectory node l : $g = g_{\text{delay}} + g_{\text{path}}$.

This is the co-design cost used by PPO transitions.

Source: `coarse_scheduler.py` (dynamic nodes), `evaluate.py` (final cost reporting).

2) What one Action means

At each decision node, PPO chooses one index over current legal branches.

One action corresponds to one branch, and that branch carries the robot scheduling/path-decision effects.

Code references: `ppo_training/environment.py` (step/reset), `ppo_training/encoding.py` (encode_actions).

3) State / Action dimensions

$\text{state_dim} = 1 + n_{\text{robots}} * (\text{ROBOT_STATE_SCALARS} + 2*(n_{\text{resources}}+1) + 3*n_{\text{resources}} + 2*n_{\text{ports}} + \text{max_route_options})$

$\text{ROBOT_STATE_SCALARS} = 10$.

$\text{action_dim} = n_{\text{robots}} * ((n_{\text{resources}}+1) + 2*(n_{\text{resources}}+1) + \text{ROBOT_ACTION_SCALARS})$

$\text{ROBOT_ACTION_SCALARS} = 4$.

Default EncodingConfig: $n_{\text{resources}}=9$, $n_{\text{ports}}=12$.

Source: `ppo_training/encoding.py` lines 47-67, 130-181.

4) Reward definition

`DecisionTreeEnv.step()`: $\text{reward} = -(\text{node_next.g} - \text{node_cur.g})$.

So reward is negative increment of accumulated cost.

Source: `ppo_training/environment.py` 79-107.

5) PPOConfig defaults (trainer.py)

$\text{discount_factor} = 1.0$

$\text{gae_lambda} = 0.95$

$\text{clip_epsilon} = 0.2$

$\text{entropy_coef} = 0.01$

$\text{value_loss_coef} = 0.5$

$\text{learning_rate} = 3e-4$

$\text{update_epochs} = 4$

$\text{minibatch_size} = 64$

$\text{max_grad_norm} = 0.5$

$\text{max_decisions_per_episode} = 200$

$\text{reward_norm_mode} = \text{none}$ (default) or `absmax`; $\text{reward_norm_minmax_eps} = 1e-12$

Source: `ppo_training/trainer.py` 25-39.

6) CLI mapping in train.py

`--learning-rate`, `--update-epochs`, `--minibatch-size`, `--entropy-coef`, `--updates`,

`--episodes-per-update`, `--max-decisions`, `--reward-norm-mode`, `--lr-schedule`, `--entropy-schedule`

map directly into PPOConfig and optimizer behavior.

Core parser and mapping also include `--fixed-case`, `--fix-shortest-paths`, `--skip-trivial-cases`.

Source: `ppo_training/train.py` lines 25-135, 337-375, 500-540.

7) GAE and normalization

$\text{delta_t} = r_{\text{t}} + \gamma * V(s_{\text{t}+1}) - V(s_{\text{t}})$

$gae_t = \delta_t + \gamma * gae_lambda * gae_{t+1}$
 $value_target_t = gae_t + V(s_t)$
Advantages are normalized by mean/std before optimization.
Source: ppo_training/rollout_buffer.py 47-63.

8) PPO objective terms

$ratio = \exp(\log_pi_new - \log_pi_old)$
 $unclipped = ratio * A_t$
 $clipped = \text{clip}(ratio, 1-\epsilon, 1+\epsilon) * A_t$
 $actor_loss = -\text{mean}(\min(unclipped, clipped))$
actor objective = actor_loss - entropy_coef * entropy
 $critic_loss = \text{mean}((V(s_t) - value_target_t)^2)$
critic objective = value_loss_coef * critic_loss
Source: ppo_training/trainer.py 329-340, 348-351.

9) Diagnostics logged every update

mean_cost, mean_reward, mean_decisions, mean_branches, actor_loss, critic_loss, entropy,
scheduled_entropy, learning_rate, approx_kl, clip_fraction, fixed_greedy_cost, elapsed_seconds.
Source: ppo_training/train.py 550-577.

10) Networks

Actor: BranchScoringActor => shared MLP over legal branches, output one logit per branch.
Critic: StateValueCritic => state-to-value MLP scalar output.
Default: 2 hidden layers, hidden_dim=128 for both.
Source: ppo_training/networks.py 15-97.

11) Training loop

collect_rollouts -> per-case collect_episode -> store RolloutBuffer -> update_with_entropy.
Each update shuffles all rollout steps and does multiple mini-batch passes.
Source: ppo_training/trainer.py 229-263 and 279-375.

12) Evaluation and gap

Greedy inference gives ppo_cost; optional exact solver gives exact_cost.
 $absolute_gap = ppo_cost - exact_cost$.
 $relative_gap = absolute_gap / \text{abs}(exact_cost)$; if exact_cost == 0 then inf when ppo_cost > 0.
Source: ppo_training/evaluate.py 79-122.

13) Data generation

ThreeByThreeCaseFactory handles random/fixed cases, initial releases, and optional shortest-path locking.
When --fix-shortest-paths is enabled, each plan has one shortest route and PPO focuses on scheduling only.
Source: ppo_training/cases.py 23-103 and 135-159.

If you need a Chinese version with native characters, I can generate it too
once a font-aware PDF method is added in this environment.